

UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM,NAGERCOIL-629 004



RECORD NOTE BOOK

**CCS355-NEURAL NETWORKS AND DEEP LEARNING
LABORATORY**

Register No :

Name :

Year/Semester :

Department :

UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM,NAGERCOIL-629 004



Register No:

*Certified that, this is the bonafide record of work done by **Mr./Ms.** of VI Semester in Computer Science and Engineering of this college, in the CCS355 -Neural Networks and Deep Learning Laboratory during academic year 2025-2026 in practical fulfilment of the requirements of the B.E Degree course of the Anna University Chennai.*

Staff-in-charge

Head of the Department

This record is submitted for the University Practical Examination held on

Internal Examiner

External Examiner

[illegible][illegible]

[illegible]

Ex No: 1

IMPLEMENT SIMPLE VECTOR ADDITION IN TENSORFLOW

Date :

AIM:

To write a python program to implement simple vector addition in TensorFlow.

ALGORITHM:

Step 1: Import the TensorFlow library.

Step 2: Define the input vectors that you want to add together. These vectors can be represented as lists, NumPy arrays, or TensorFlow tensors.

Step 3: Convert the input vectors into TensorFlow constant tensors using the `tf.constant` function. This step ensures that the input data is compatible with TensorFlow operations.

Step 4: Perform addition operation using the `tf.add` function to add the two input tensors together. This function performs element-wise addition, adding corresponding elements from each tensor.

Step 5: Run the TensorFlow Session.

Step 6: Retrieve the result using the `numpy()` method to convert the TensorFlow tensor to a NumPy array.

Step 7: Output the result of the addition operation, which represents the sum of the two input vectors.

PROGRAM:

```
import tensorflow as tf

scalar = tf.constant(7)
print(scalar.ndim)

vector = tf.constant([10, 10])

matrix = tf.constant([[1, 2], [3, 4]])
print(matrix)

print("The number of dimensions of a matrix is: " + str(matrix.ndim))

matrix = tf.constant([[1, 2], [3, 4]])
matrix1 = tf.constant([[2, 4], [6, 8]])

print("Addition of two matrices:")

print(matrix + matrix1)
```

OUTPUT:

```
tf.Tensor(  
[[1 2]  
 [3 4]], shape=(2, 2), dtype=int32)  
The number of dimensions of a matrix is: 2  
Addition of two matrices:  
tf.Tensor(  
[[ 3  6]  
 [ 9 12]], shape=(2, 2), dtype=int32)
```

RESULT:

Thus the program for Simple Vector Addition in TensorFlow was executed successfully.

Ex No: 2

Date :

IMPLEMENT A REGRESSION MODEL IN KERAS

AIM:

To write a python program to implement a regression model in Keras.

ALGORITHM:

Step 1: Import libraries NumPy and TensorFlow libraries are imported. Specifically, TensorFlow's Keras API is imported to define and train the neural network model.

Step 2: Generate Random data for regression is generated using NumPy. X represents the features, and y represents the labels. The labels (y) are generated based on a linear relationship with some added noise.

Step 3: Define a sequential model is defined using Keras. It consists of two dense layers. The first layer has 10 neurons with ReLU activation function, and it expects input of shape (1,). The second layer has 1 neuron, which is the output neuron for regression.

Step 4: The model is compiled using the Adam optimizer and mean squared error loss function, which are commonly used for regression tasks.

Step 5: The model is trained on the generated data for 100 epochs with a batch size of 32. The training process aims to minimize the mean squared error loss.

Step 6: Once the model is trained, predictions are made on the same data X used for training.

Step 7: A loop is used to print the predictions made by the model along with the corresponding true labels(y). This allows for a visual comparison of the model's performance.

PROGRAM:

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
np.random.seed(0)
X = np.random.rand(100, 1)
y = 2 * X.squeeze() + 1 + np.random.randn(100) * 0.1
model=keras.Sequential([layers.Dense(10,activation='relu',input_shape=(1,)),
layers.Dense(1)])
model.compile(optimizer='adam', loss='mse')
model.fit(X, y, epochs=100, batch_size=32, verbose=0)
```

```
predictions = model.predict(X)

for i in range(5):

    print("Predicted:", predictions[i][0], "\tTrue:", y[i])
```

OUTPUT:

```
4/4 ————— 0s 15ms/step
Predicted: 1.9927089    True: 1.9811120237763138
Predicted: 2.3151395    True: 2.520461381440258
Predicted: 2.097262     True: 2.252092996116334
Predicted: 1.985092     True: 1.9361419973660712
Predicted: 1.7501557    True: 1.9961348180573695
```

RESULT:

Thus the program for a Regression Model in Keras was executed successfully.

Ex No:3

Date :

IMPLEMENT A PERCEPTRON IN TENSORFLOW/KERAS ENVIRONMENT

AIM:

To write a python program to implement a perceptron in TensorFlow/Keras Environment.

ALGORITHM:

- Step 1: Start
- Step 2: Import required libraries (NumPy, TensorFlow/Keras)
- Step 3: Prepare input data (X) and output labels (y) for OR operation
- Step 4: Create a Sequential model with one Dense layer (sigmoid activation)
- Step 5: Compile the model using Adam optimizer and binary cross-entropy loss
- Step 6: Train the model with the dataset
- Step 7: Evaluate the model and print accuracy
- Step 8: Predict output using the trained model
- Step 9: Stop

PROGRAM:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Generate some example data for a logical OR operation

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

y = np.array([0, 1, 1, 1])

# Define the perceptron model

model = Sequential([Dense(1, input_shape=(2,), activation='sigmoid', use_bias=True)])

# Compile the model

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Train the model

model.fit(X, y, epochs=1000, verbose=0)

# Evaluate the model

loss, accuracy = model.evaluate(X, y)

print("Loss:", loss)

print("Accuracy:", accuracy)

# Make predictions

predictions = model.predict(X)

print("Predictions:", predictions.flatten())
```

OUTPUT:

1/1 ————— 0s 103ms/step - accuracy: 0.7500 - loss: 0.5243

Loss: 0.5242975950241089

Accuracy: 0.75

1/1 ————— 0s 40ms/step

Predictions: [0.65719175 0.56547964 0.82771665 0.7653355]

RESULT:

Thus the program for perceptron in TensorFlow/Keras Environment was done successfully.

Ex No:4

Date :

IMPLEMENT A FEED FORWARD NETWORK IN TENSORFLOW/KERAS

AIM:

To write a python program to implement a Feed Forward Neural Network using TensorFlow/keras.

ALGORITHM:

Step 1: Import necessary libraries (NumPy, TensorFlow, Keras, Matplotlib).

Step 2: Set random seed for reproducibility.

Step 3: Load the MNIST dataset.

Step 4: Split the dataset into training, validation, and test sets.

Step 5: Reshape input images into 1D arrays.

Step 6: Normalize pixel values to the range 0–1.

Step 7: Convert labels into one-hot encoded format.

Step 8: Build the Feed Forward Neural Network using Sequential model.

Step 9: Compile and train the model using training and validation data.

Step 10: Evaluate the model on test data and plot accuracy.

PROGRAM:

```
import tensorflow as tf

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Flatten

# Load dataset

(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
# Normalize data

X_train = X_train / 255.0

X_test = X_test / 255.0

# Build model

model = Sequential([

    Flatten(input_shape=(28, 28)),

    Dense(128, activation='relu'),

    Dense(10, activation='softmax')

])

# Compile model

model.compile(optimizer='adam',

              loss='sparse_categorical_crossentropy',

              metrics=['accuracy'])

# Train model (store history)

history = model.fit(X_train, y_train, epochs=5, validation_split=0.2)

# Evaluate model

loss, acc = model.evaluate(X_test, y_test)

print("Accuracy:", acc)

# Plot accuracy

plt.plot(history.history['accuracy'], label='train')

plt.plot(history.history['val_accuracy'], label='validation')

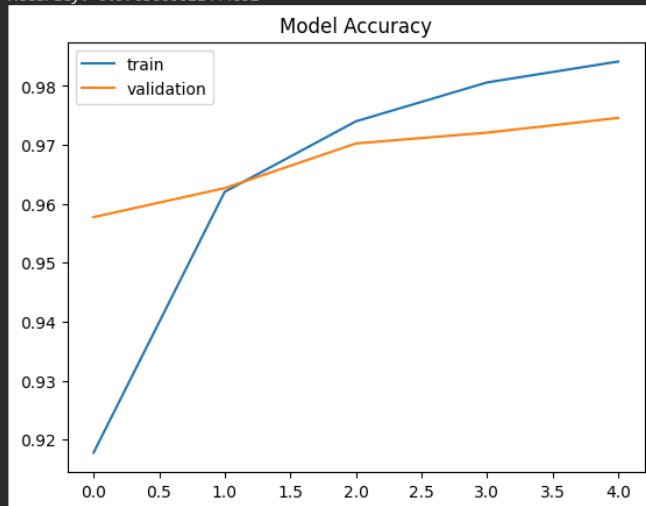
plt.legend()

plt.title("Model Accuracy")

plt.show()
```

OUTPUT:

```
Epoch 1/5  
1500/1500 ————— 7s 4ms/step - accuracy: 0.9178 - loss: 0.2895 - val_accuracy: 0.9578 - val_loss: 0.1521  
Epoch 2/5  
1500/1500 ————— 8s 5ms/step - accuracy: 0.9620 - loss: 0.1295 - val_accuracy: 0.9627 - val_loss: 0.1265  
Epoch 3/5  
1500/1500 ————— 7s 4ms/step - accuracy: 0.9740 - loss: 0.0880 - val_accuracy: 0.9703 - val_loss: 0.0980  
Epoch 4/5  
1500/1500 ————— 8s 5ms/step - accuracy: 0.9806 - loss: 0.0664 - val_accuracy: 0.9721 - val_loss: 0.0888  
Epoch 5/5  
1500/1500 ————— 7s 5ms/step - accuracy: 0.9841 - loss: 0.0513 - val_accuracy: 0.9746 - val_loss: 0.0855  
313/313 ————— 1s 2ms/step - accuracy: 0.9763 - loss: 0.0764  
Accuracy: 0.9763000011444092
```



RESULT:

Thus the program for a Feed Forward neural network using TensorFlow/Keras was executed successfully.

Ex No : 5

IMPLEMENT AN IMAGE CLASSIFIER USING CNN IN TENSORFLOW/KERAS

Date :

AIM:

To write the python program to implement an image classifier using CNN in tensorflow/keras.

ALGORITHM:

Step 1 : Import necessary libraries.

Step 2 : Load and prepare the CIFAR-10 dataset.

Step 3 : Define class names for the CIFAR-10 dataset.

Step 4 : Visualize the first 25 images from the training set.

Step 5 : Define the Convolutional Neural Network (CNN) model.

Step 6 : Compile the model and train the model on the training data.

Step 7 : Plot the training history (accuracy and epochs) and evaluate the model on the test data.

PROGRAM:

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models

import matplotlib.pyplot as plt

(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

X_train, X_test = X_train/255.0, X_test/255.0

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))

for i in range(25):

    plt.subplot(5,5,i+1)

    plt.xticks([]);

    plt.yticks([]);

    plt.grid(False)

    plt.imshow(X_train[i])

    plt.xlabel(class_names[y_train[i][0]])

plt.show()
```

```
model=models.Sequential([layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)),layers.MaxPooling2D(2,2),layers.Conv2D(64,(3,3),activation='relu'),layers.MaxPooling2D(2,2),layers.Conv2D(64,(3,3),activation='relu'),layers.Flatten(),layers.Dense(64,activation='relu'),layers.Dense(10)])
```

```
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])
```

```
history = model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))
```

```
plt.plot(history.history['accuracy'], label='train')
```

```
plt.plot(history.history['val_accuracy'], label='val')
```

```
plt.xlabel('Epoch'); plt.ylabel('Accuracy');
```

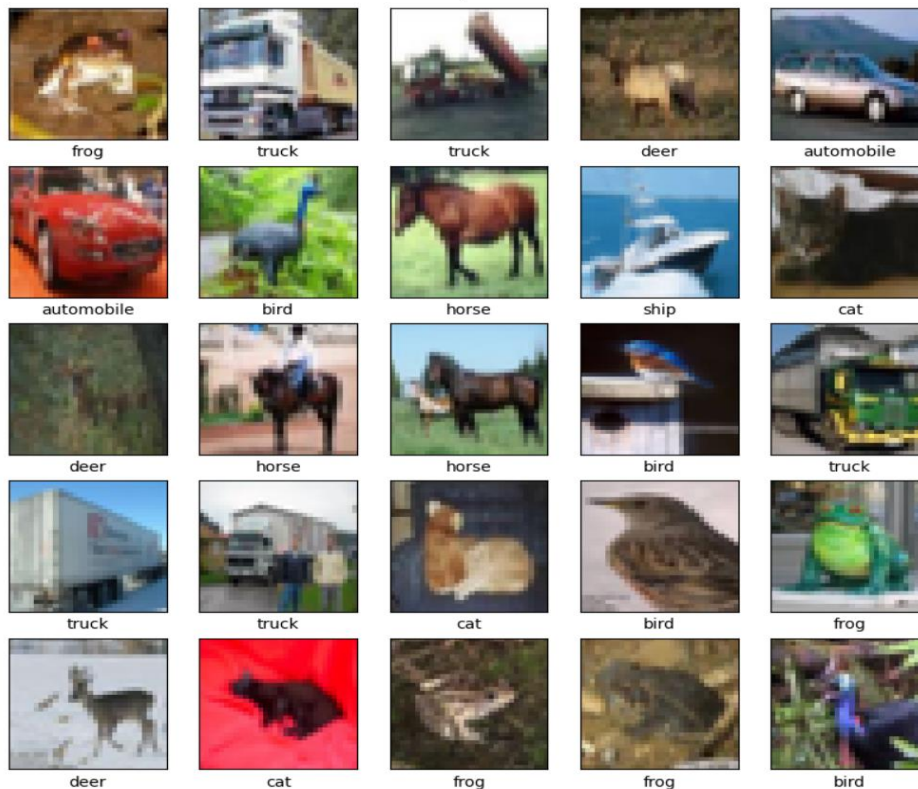
```
plt.legend();
```

```
plt.show()
```

```
print("Test accuracy:", model.evaluate(X_test, y_test, verbose=0)[1])
```

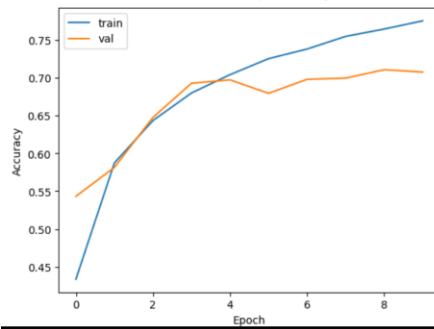
OUTPUT:

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 4s 0us/step



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, pre
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/10
1563/1563 — 81s 50ms/step - accuracy: 0.4336 - loss: 1.5467 - val_accuracy: 0.5432 - val_loss: 1.2613
Epoch 2/10
1563/1563 — 74s 47ms/step - accuracy: 0.5874 - loss: 1.1616 - val_accuracy: 0.5815 - val_loss: 1.1879
Epoch 3/10
1563/1563 — 76s 49ms/step - accuracy: 0.6434 - loss: 1.0121 - val_accuracy: 0.6475 - val_loss: 1.0042
Epoch 4/10
1563/1563 — 73s 47ms/step - accuracy: 0.6798 - loss: 0.9179 - val_accuracy: 0.6927 - val_loss: 0.8855
Epoch 5/10
1563/1563 — 73s 47ms/step - accuracy: 0.7040 - loss: 0.8489 - val_accuracy: 0.6972 - val_loss: 0.8669
Epoch 6/10
1563/1563 — 82s 47ms/step - accuracy: 0.7252 - loss: 0.7939 - val_accuracy: 0.6794 - val_loss: 0.9166
Epoch 7/10
1563/1563 — 73s 47ms/step - accuracy: 0.7379 - loss: 0.7519 - val_accuracy: 0.6979 - val_loss: 0.8665
Epoch 8/10
1563/1563 — 72s 46ms/step - accuracy: 0.7546 - loss: 0.7081 - val_accuracy: 0.6995 - val_loss: 0.8685
Epoch 9/10
1563/1563 — 74s 47ms/step - accuracy: 0.7643 - loss: 0.6727 - val_accuracy: 0.7106 - val_loss: 0.8505
Epoch 10/10
1563/1563 — 73s 47ms/step - accuracy: 0.7752 - loss: 0.6406 - val_accuracy: 0.7075 - val_loss: 0.8540
```



RESULT:

Thus the python program for image classifier using CNN in tensorflow/keras was executed successfully.

Ex No : 6

IMPROVE THE DEEP LEARNING MODE BY TUNING HYPER PRAMETER

Date :

AIM:

To write a python program to improve the deep learning model by fine tuning hyper parameters.

ALGORITHM:

Step 1 : Import necessary libraries.

Step 2 : Generate a synthetic dataset.

Step 3 : Define parameter distribution for hyperparameter tuning.

Step 4 : Initialize DecisionTreeClassifier and run RandomizedSearchCV with param_dist on (X,Y).

Step 5 : Print the best parameters and best score.

PROGRAM:

```
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

X, y = make_classification(n_samples=1000, n_features=20, n_informative=10, n_classes=2,
    random_state=42)

param_dist = {
    "max_depth": [3, None],
    "max_features": randint(1, 9),
    "min_samples_leaf": randint(1, 9),
    "criterion": ["gini", "entropy"]
}

tree_cv = RandomizedSearchCV(DecisionTreeClassifier(), param_dist, cv=5).fit(X, y)

print("Tuned Parameters:", tree_cv.best_params_)
print("Best Score:", tree_cv.best_score_)
```

OUTPUT:

```
*** Tuned Parameters: {'criterion': 'gini', 'max_depth': None, 'max_features': 8, 'min_samples_leaf': 4}
Best Score: 0.8480000000000001
```

RESULT:

Thus the program for deep learning model by fine tuning hyper parameters was executed successfully.

Ex No: 7

Date :

IMPLEMENT A TRANSFER LEARNING CONCEPT IN IMAGE CLASSIFICATION

AIM:

To write a python program to implement a transfer learning concept in image classification

ALGORITHM:

Step1: Import required libraries such as TensorFlow, Keras, NumPy, and Image preprocessing modules.

Step2: Load the dataset from the given directory using ImageDataGenerator.

Step3: Rescale pixel values and split the dataset into training and validation sets.

Step4: Load the pre-trained MobileNetV2 model without the top (output) layer.

Step5: Freeze the base model layers to retain learned features.

Step6: Add custom layers (Flatten and Dense) for classification.

Step7: Compile the model using Adam optimizer and binary cross-entropy loss.

Step8: Train the model using training data and validate using validation data.

Step9: Load a test image and preprocess it.

Step10: Predict the class of the image using the trained model.

Step11: Display the prediction result as Cat or Dog.

PROGRAM:

```
import tensorflow as tf
import numpy as np
from tensorflow.keras import datasets, layers, models

# Load dataset
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

# Keep only cat (3) and dog (5)
train_filter = np.where((y_train == 3) | (y_train == 5))[0]
test_filter = np.where((y_test == 3) | (y_test == 5))[0]
X_train, y_train = X_train[train_filter], y_train[train_filter]
X_test, y_test = X_test[test_filter], y_test[test_filter]

# Convert labels: cat=0, dog=1
y_train = (y_train == 5).astype(int)
```

```

y_test = (y_test == 5).astype(int)
# Normalize
X_train, X_test = X_train/255.0, X_test/255.0
# Model
model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)),
    layers.MaxPooling2D(2,2),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),
    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(1,activation='sigmoid')
])

# Compile
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train
model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))

# Test prediction
pred = model.predict(X_test[:1])
print("Prediction:", "DOG" if pred[0][0] > 0.5 else "CAT")

```

OUTPUT:

```

Found 800 images belonging to 2 classes.
Found 200 images belonging to 2 classes.
Epoch 1/2
25/25 ————— 41s 1s/step - accuracy: 0.9800 - loss: 0.0408 - val_accuracy: 1.0000 - val_loss: 1.6370e-05
Epoch 2/2
25/25 ————— 40s 1s/step - accuracy: 1.0000 - loss: 8.6814e-04 - val_accuracy: 1.0000 - val_loss: 7.4274e-06
1/1 ————— 1s 1s/step
Prediction: CAT

```

RESULT:

Thus the python program for a transfer learning concept in image classification was executed successfully.

Ex No:8

Date :

USING A PRE TRAINED MODEL ON KERAS FOR TRANSFER LEARNING

AIM:

To write a python program to use a pretrained model on keras for transfer learning.

ALGORITHM:

Step1: Import the pre-trained VGG16 model from Keras applications.

Step2: Load the VGG16 model with ImageNet weights excluding the top layer.

Step3: Freeze the initial layers of the model and allow the last few layers to be trainable (fine-tuning).

Step4: Add custom layers such as Flatten and Dense for binary classification.

Step5: Compile the model using Adam optimizer and binary cross-entropy loss function.

Step6: Train the model using the training dataset and validate using validation data.

Step7: Load a test image and preprocess it (resize, normalize, reshape).

Step8: Use the trained model to predict the class of the image.

Step9: Display the predicted output as Cat or Dog.

PROGRAM:

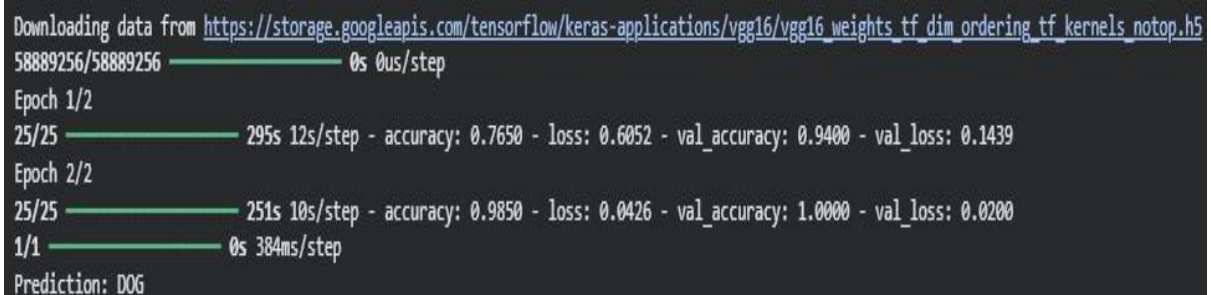
```
import tensorflow as tf
import numpy as np
from tensorflow.keras.applications import VGG16
# Load dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()
# Keep only cat(3) and dog(5)
train = np.where((y_train==3)|(y_train==5))[0]
test = np.where((y_test==3)|(y_test==5))[0]
X_train, y_train = X_train[train], (y_train[train]==5).astype(int)
X_test, y_test = X_test[test], (y_test[test]==5).astype(int)
# Reduce dataset size (for speed)
X_train = X_train[:2000]
y_train = y_train[:2000]
# Resize + normalize
```

```

X_train = tf.image.resize(X_train,(64,64))/255.0
X_test = tf.image.resize(X_test,(64,64))/255.0
# Load VGG16
base = VGG16(weights='imagenet', include_top=False, input_shape=(64,64,3))
base.trainable = False
# Build model
model = tf.keras.Sequential([
    base,
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
# Compile
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Train
model.fit(X_train, y_train, epochs=2, validation_data=(X_test, y_test))
# Predict
pred = model.predict(X_test[:1])
print("Prediction:", "DOG" if pred[0][0] > 0.5 else "CAT")

```

OUTPUT:



```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ————— 0s 0us/step
Epoch 1/2
25/25 ————— 295s 12s/step - accuracy: 0.7650 - loss: 0.6052 - val_accuracy: 0.9400 - val_loss: 0.1439
Epoch 2/2
25/25 ————— 251s 10s/step - accuracy: 0.9850 - loss: 0.0426 - val_accuracy: 1.0000 - val_loss: 0.0200
1/1 ————— 0s 384ms/step
Prediction: DOG

```

RESULT:

Thus the python program for pre-trained model on keras for transfer learning was executed successfully.

Ex No: 9

PERFORM SENTIMENT ANALYSIS USING RNN

Date :

AIM:

To write a python program to perform sentiment analysis using RNN.

ALGORITHM:

- Step 1: Import required libraries (tensorflow, dataset, preprocessing, layers).
- Step 2: Define parameters like vocabulary size and maximum sequence length.
- Step 3: Load the IMDB dataset and split into training and testing data.
- Step 4: Pad all sequences to the same length using pad_sequences.
- Step 5: Build the RNN model using Embedding → SimpleRNN → Dense layers.
- Step 6: Compile and train the model with binary crossentropy and Adam optimizer.
- Step 7: Take user input, encode the review, predict sentiment, and display result.

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
import numpy as np
vocab_size = 10000
maxlen = 500
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)
model = Sequential()
model.add(Embedding(vocab_size, 32))
```

```

model.add(tf.keras.layers.SimpleRNN(32))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_data=(x_test, y_test))
word_index = imdb.get_word_index()
index_word = {v+3: k for k, v in word_index.items()}
index_word[0] = "<PAD>"
index_word[1] = "<START>"
index_word[2] = "<UNK>"
index_word[3] = "<UNUSED>"
def review_encode(s):
    encoded = [1] # start token
    for word in s.lower().split():
        if word in word_index and word_index[word] + 3 < vocab_size:
            encoded.append(word_index[word] + 3)
        else:
            encoded.append(2) # unknown word
    return pad_sequences([encoded], maxlen=maxlen)
while True:
    review = input("Enter a movie review (type 'exit' to quit): ")
    if review.lower() == "exit":
        break
    encoded_review = review_encode(review)
    prediction = model.predict(encoded_review)[0][0]
    sentiment = "Positive Sentiment" if prediction > 0.5 else "Negative Sentiment"
    print(sentiment)

```


OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 — 0s 0us/step
Model: "sequential"

```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
391/391 — 67s 165ms/step - accuracy: 0.6928 - loss: 0.5652 - val_accuracy: 0.8295 - val_loss: 0.3895
Epoch 2/10
391/391 — 93s 193ms/step - accuracy: 0.8278 - loss: 0.3914 - val_accuracy: 0.8246 - val_loss: 0.3970
Epoch 3/10
391/391 — 81s 192ms/step - accuracy: 0.8922 - loss: 0.2689 - val_accuracy: 0.8390 - val_loss: 0.4331
Epoch 4/10
391/391 — 73s 169ms/step - accuracy: 0.9024 - loss: 0.2442 - val_accuracy: 0.7729 - val_loss: 0.5039
Epoch 5/10
391/391 — 67s 172ms/step - accuracy: 0.9572 - loss: 0.1212 - val_accuracy: 0.8261 - val_loss: 0.4545
Epoch 6/10
391/391 — 76s 195ms/step - accuracy: 0.9799 - loss: 0.0653 - val_accuracy: 0.8340 - val_loss: 0.5281
Epoch 7/10
391/391 — 76s 194ms/step - accuracy: 0.9921 - loss: 0.0311 - val_accuracy: 0.8209 - val_loss: 0.6132
Epoch 8/10
391/391 — 64s 164ms/step - accuracy: 0.9816 - loss: 0.0536 - val_accuracy: 0.7740 - val_loss: 0.6864
Epoch 9/10
391/391 — 66s 168ms/step - accuracy: 0.9820 - loss: 0.0531 - val_accuracy: 0.8226 - val_loss: 0.6978
Epoch 10/10
391/391 — 67s 171ms/step - accuracy: 0.9980 - loss: 0.0099 - val_accuracy: 0.8324 - val_loss: 0.7560
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 — 0s 0us/step
Enter a movie review (type 'exit' to quit): I like cat
1/1 — 0s 219ms/step
Positive Sentiment
Enter a movie review (type 'exit' to quit): I dont like dog
1/1 — 0s 53ms/step
Negative Sentiment
Enter a movie review (type 'exit' to quit): exit
```

RESULT:

Thus the program for sentiment analysis using RNN was executed successfully.

Ex No: 10

IMPLEMENT AN LSTM BASED AUTOENCODER IN TENSORFLOW/KERAS

Date :

AIM:

To write a python program to implement an LSTM based auto-encoder tensorflow/keras.

ALGORITHM:

Step 1: Import required libraries such as NumPy, TensorFlow, and Keras modules for model building.

Step 2: Generate sample dataset using NumPy with shape (1000, 10, 1) and define input shape.

Step 3: Define the input layer using Input(shape=(10,1)).

Step 4: Build the encoder using an LSTM layer to compress the input data and use RepeatVector to prepare for decoding.

Step 5: Build the decoder using LSTM (with return_sequences=True) followed by a TimeDistributed Dense layer.

Step 6: Create the autoencoder model, compile it using Adam optimizer and Mean Squared Error loss, then train the model with input data.

Step 7: Extract the encoder model and define decoder layers for reconstructing the input from encoded representation.

PROGRAM:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Model, Sequential

from tensorflow.keras.layers import LSTM, RepeatVector, TimeDistributed, Dense, Input

n_samples = 1000

n_timesteps = 10

n_features = 1

X = np.random.randn(n_samples, n_timesteps, n_features)

model = Sequential([

    Input(shape=(n_timesteps, n_features)),
```

```

LSTM(4, name="lstm_6"),

RepeatVector(n_timesteps, name="repeat_vector_3"),

LSTM(4, return_sequences=True, name="lstm_7"),

TimeDistributed(Dense(n_features), name="time_distributed_3")

])

model.compile(optimizer='adam', loss='mse')

model.summary()

model.fit(X, X, epochs=5, batch_size=32, verbose=0)

encoder_model = Model(inputs=model.inputs, outputs=model.get_layer("lstm_6").output)

while True:

    user_input = input("\nEnter a sequence of 10 numbers separated by spaces (type 'exit' to quit):
    ")

    if user_input.lower() == 'exit':

        break

    try:

        raw_seq = np.array([float(i) for i in user_input.split()])

        if len(raw_seq) != 10:

            print("Please enter exactly 10 numbers.")

            continue

        test_input = raw_seq.reshape((1, n_timesteps, n_features))

        encoded_output = encoder_model.predict(test_input)

        decoded_output = model.predict(test_input)

        print(f'Original Sequence: {test_input}')

        print(f'Encoded Sequence: {encoded_output}')

        print(f'Decoded Sequence: {decoded_output}')

```

except ValueError:

```
print("Invalid input.")
```

OUTPUT:

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
lstm_6 (LSTM)	(None, 4)	96
repeat_vector_3 (RepeatVector)	(None, 10, 4)	0
lstm_7 (LSTM)	(None, 10, 4)	144
time_distributed_3 (TimeDistributed)	(None, 10, 1)	5

Total params: 245 (980.00 B)
Trainable params: 245 (980.00 B)
Non-trainable params: 0 (0.00 B)

Enter a sequence of 10 numbers separated by spaces (type 'exit' to quit): 3 5 7 4 77 4 5 9 1 22
1/1 _____ 0s 321ms/step
1/1 _____ 1s 787ms/step
Original Sequence: [[[3.]
[5.]
[7.]
[4.]
[77.]
[4.]
[5.]
[9.]
[1.]
[22.]]]

Encoded Sequence: [[0.847463 0.13571322 0.56329614 0.00849997]]
Decoded Sequence: [[[0.24665518]
[0.4248987]
[0.5403066]
[0.6110082]
[0.65400654]
[0.68062943]
[0.69757485]
[0.7086819]
[0.71616673]
[0.7213373]]]]

Enter a sequence of 10 numbers separated by spaces (type 'exit' to quit): exit

RESULT:

Thus the program for an LSTM (Long short-Term Memory) based auto-encoder was executed successfully.

Ex No:11

Date :

IMAGE GENERATION USING GAN

AIM:

To write a python program to implement image generation using GAN.

ALGORITHM:

Step1: Import TensorFlow, Keras, and NumPy.

Step2: Load the MNIST dataset and normalize pixel values to the range $[-1, 1]$ and add a channel dimension.

Step3: Create the generator model using a Sequential model with layers for dense, reshape, and transpose convolution operations.

Step4: Create the discriminator model using another Sequential model with convolution layers followed by a dense layer for binary classification.

Step5: Compile the discriminator model with binary cross-entropy loss and the Adam optimizer.

Step6: Set discriminator.trainable = False to freeze the discriminator's weights during GAN training.

Step7: Create the GAN model by connecting the generator and discriminator in a sequential manner.

Step8: Compile the GAN model with binary cross-entropy loss and the Adam optimizer.

Step9: END.

PROGRAM:

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt

# Load data
(X_train, _), (_, _) = keras.datasets.mnist.load_data()
X_train = (X_train - 127.5) / 127.5
X_train = X_train.reshape(-1, 28, 28, 1)

# Generator
generator = keras.Sequential([
```

```

keras.layers.Dense(128, input_dim=100),
keras.layers.LeakyReLU(0.2),
keras.layers.Dense(784, activation='tanh'),
keras.layers.Reshape((28,28,1))
])

# Discriminator
discriminator = keras.Sequential([
    keras.layers.Flatten(input_shape=(28,28,1)),
    keras.layers.Dense(128),
    keras.layers.LeakyReLU(0.2),
    keras.layers.Dense(1, activation='sigmoid')
])

discriminator.compile(loss='binary_crossentropy', optimizer='adam')

# GAN
discriminator.trainable = False
gan = keras.Sequential([generator, discriminator])
gan.compile(loss='binary_crossentropy', optimizer='adam')

# Training
for epoch in range(10):
    idx = np.random.randint(0, X_train.shape[0], 32)
    real = X_train[idx]
    noise = np.random.normal(0,1,(32,100))
    fake = generator.predict(noise, verbose=0)
    d_loss_real = discriminator.train_on_batch(real, np.ones((32,1)))
    d_loss_fake = discriminator.train_on_batch(fake, np.zeros((32,1)))
    g_loss = gan.train_on_batch(noise, np.ones((32,1)))
    print(f'Epoch {epoch+1}, D Loss: {d_loss_real}, G Loss: {g_loss}')

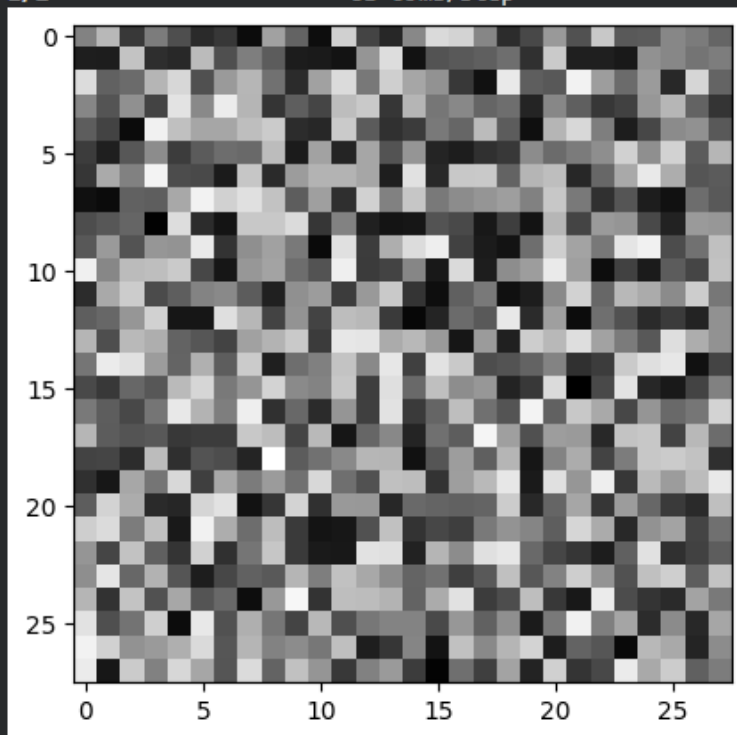
# Generate image
noise = np.random.normal(0,1,(1,100))

```

```
img = generator.predict(noise)
plt.imshow(img.reshape(28,28), cmap='gray')
plt.show()
```

OUTPUT:

```
*** Epoch 1, D Loss: 0.9283316135406494, G Loss: 0.7106399536132812
Epoch 2, D Loss: 0.886293888092041, G Loss: 0.5055531859397888
Epoch 3, D Loss: 1.002648949623108, G Loss: 0.3872058093547821
Epoch 4, D Loss: 1.1353576183319092, G Loss: 0.31057846546173096
Epoch 5, D Loss: 1.2876715660095215, G Loss: 0.257817804813385
Epoch 6, D Loss: 1.4569625854492188, G Loss: 0.2191208153963089
Epoch 7, D Loss: 1.6090073585510254, G Loss: 0.1896377056837082
Epoch 8, D Loss: 1.7737830877304077, G Loss: 0.16724197566509247
Epoch 9, D Loss: 1.9198198318481445, G Loss: 0.14911526441574097
Epoch 10, D Loss: 2.0861117839813232, G Loss: 0.13451939821243286
1/1 ————— 0s 69ms/step
```



RESULT:

Thus the program for image generation using GAN (Generative Adversarial Network) was executed successfully.

Ex No:12

Date :

IMPLEMENT ANY SIMPLE REINFORCEMENT ALGORITHM FOR AN NLP PROBLEM

AIM:

To write a python program to implement any simple reinforcement algorithm for an NLP problem.

ALGORITHM:

Step1: Initialize Q-learning parameters: states (inputs), actions (responses), Q-table, alpha, gamma, epsilon.

Step2: Define reward function: `get_reward(state, action)` to return reward for correct or incorrect response.

Step3: Implement Q-learning algorithm:

- a. Define training loop for number of episodes.
- b. Select state and choose action using ϵ -greedy method.
- c. Update Q-table using Q-learning formula.

Step4: Define response generation: create function to select best action using Q-table.

Step5: Train the Q-learning model: run training for given number of episodes.

Step6: Start interaction: take user input and display the generated response.

PROGRAM:

```
import numpy as np
# States (user inputs)
states = ["hi", "bye", "thanks"]
# Actions (responses)
actions = ["hello", "goodbye", "welcome"]
# Q-table (3x3)
Q = np.zeros((3,3))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
# Reward function
def get_reward(state, action):
    if state == 0 and action == 0: return 1 # hi -> hello
    if state == 1 and action == 1: return 1 # bye -> goodbye
    if state == 2 and action == 2: return 1 # thanks -> welcome
    return -1
```



```

# Training
for episode in range(100):
    state = np.random.randint(0,3)
    if np.random.rand() < epsilon:
        action = np.random.randint(0,3)
    else:
        action = np.argmax(Q[state])
    reward = get_reward(state, action)
    Q[state, action] = Q[state, action] + alpha * (reward + gamma * np.max(Q[state]) - Q[state,
    action])
# Testing (NLP interaction)
print("Enter: hi / bye / thanks")
user = input()
if user in states:
    state = states.index(user)
    action = np.argmax(Q[state])
    print("Bot:", actions[action])
else:
    print("Unknown input")

```

OUTPUT:

```

.. Enter: hi / bye / thanks
   hi
   Bot: hello

```

RESULT:

Thus the python program for reinforcement algorithm for an NLP problem was executed successfully.